

SeuGado — Leo: o bot do Telegram

A única interface do produtor: plano, respostas, alturas e atualizações

27/09/2026

Sua parte em uma frase: no MVP, **o bot é o SeuGado inteiro para o produtor**. Ele não usa site nenhum: recebe o plano da semana no dia e hora que escolheu, diz com um toque se fez, não fez ou fez diferente, corrige a altura dos piquetes, e decide se aceita um plano atualizado quando chegam imagens novas do satélite.

Você entrega: `delivery/` (texto, canal Telegram, envio, respostas, lembretes, plano candidato, conversa), `api/rotas_telegram.py` (webhook) e `scripts/telegram_polling.py` (modo de desenvolvimento).

Prazo: PR aberto até domingo à noite.

1. O SeuGado em um minuto

O gado é dividido em **lotes**, e o pasto em **piquetes**. O capim tem uma altura certa para o lote entrar e uma altura em que ele precisa sair. O sistema estima as alturas por satélite e clima todo dia, e o otimizador (João) monta o **plano da semana**. Você entrega esse plano e traz de volta a realidade: o que de fato aconteceu no campo. **A resposta “fiz diferente” é o dado mais valioso do sistema:** é onde o produtor discordou da máquina (ADR-007).

Regras que valem para você (ADR-025):

- **Só o produtor confirma.** Nada é dado como feito sem a resposta dele. Enquanto ele não responde, o sistema considera que o lote **não** saiu do lugar e lembra uma vez por dia.
- **O plano semanal** chega no dia e hora escolhidos para a fazenda (quem dispara é a rotina do Leandro, que chama a sua `enviar_plano`).
- **Plano candidato:** se no meio da semana o satélite traz dado novo e o plano muda em algo que o produtor ainda não fez, você pergunta se ele quer ver as mudanças. Ele pode manter o plano dele ou trocar pelo novo.
- **Confirmar custa um toque** (ADR-004). Tudo é por botão, exceto a altura, que é um número.

Palavras que você vai usar

Termo	O que é
Plano vigente	O plano que o produtor está seguindo
Plano candidato	Plano novo, oferecido ao produtor, que ele aceita ou recusa
Movimentação	“Mover o lote X do piquete A para o B no dia D”. Tem um <code>id</code> único
Fiz	Fez como recomendado → evento <code>manejo_confirmado</code>
Não fiz	Não fez → evento <code>manejo_recusado</code> → recalcular
Fiz diferente	Levou para outro piquete ou em outro dia → <code>manejo_divergente</code> → recalcular

Termo	O que é
Movimentação avulsa	Mudou um lote sem recomendação → <code>manejo_confirmado</code> com id novo → recalcular
Altura medida	O produtor mede o capim com régua e manda o número → evento <code>altura_medida</code>
Evento	Registro imutável de algo que aconteceu. Nunca se edita o passado

Entradas e saídas: o que você recebe, de quem, e o que entrega

Esta seção é o seu contrato com o resto da equipe. **Nomes, tipos e formatos são exatamente estes**; se algo aqui parecer faltar ou estar errado, fale com o Kauê antes de inventar outro formato.

Resumo

	O quê	De quem / para quem	Como chega / sai
Recebe	plano para enviar	rotina do Leandro (plano gerado pelo João)	<code>enviar_plano(conn, plano, atualizado)</code>
Recebe	plano candidato + o que mudou	rotina do Leandro	<code>avisar_plano_candidato(conn, plano, diferencas)</code>
Recebe	pedido de lembrete diário	rotina do Leandro	<code>lembrar_pendentes(conn, fazenda_id, hoje)</code>
Recebe	mensagens e cliques do produtor	Telegram	POST /telegram/webhook (JSON do Telegram, campos abaixo)
Usa	planos gravados	João	<code>carregar_plano_atual</code> , <code>carregar_candidato</code> , <code>carregar_plano</code> , <code>status_do_plano</code> , <code>promover_candidato</code> , <code>descartar_plano</code> , <code>ids_respondidos</code> (<code>persistencia/planos.py</code>) e <code>comparar_planos</code> (<code>planner/comparacao.py</code>)
Usa	nomes e posições de lotes e piquetes	banco (derivadas do Kauê)	SQL em <code>estado_lote</code> , <code>estado_piquete</code> , <code>fazenda</code>
Entrega	respostas do produtor como eventos	banco → Kauê (estado) → João (próximo plano)	<code>registrar_evento</code> + <code>reconstruir_projecao</code> + <code>commit</code>
Chama	recálculo imediato	Leandro	<code>executar_ciclo(conn, fazenda_id, ingerir_satelite=False, atualizado=True) -> PlanoManejo</code> (já salva como vigente e envia)

Assinaturas exatas que outros chamam

```
# src/seugado/delivery/envio.py (conn: psycopg.Connection[Any])
def enviar_plano(conn, plano: PlanoManejo, atualizado: bool = False,
                 canal: Canal | None = None) -> bool: ...
```

```

# True = enviou; False = fazenda sem Telegram vinculado (não é erro)
def avisar_plano_candidato(conn, plano: PlanoManejo, diferencas: tuple[DiferencaLote,
...],
                        canal: Canal | None = None) -> bool: ...

# src/seugado/delivery/confirmacao.py
def lembrar_pendentes(conn, fazenda_id: UUID, hoje: date,
                    canal: Canal | None = None) -> int: ...
    # quantas movimentações pendentes foram lembradas; NÃO faz commit

# src/seugado/api/rotas_telegram.py
# POST /telegram/webhook
# cabeçalho obrigatório: X-Telegram-Bot-API-Secret-Token = TELEGRAM_WEBHOOK_SECRET
(senão 403)
# corpo: o "Update" do Telegram (JSON)
# resposta imediata: {"ok": true} (o processamento roda em BackgroundTasks)

```

O que chega do Telegram (só os campos que você usa)

```

// mensagem de texto
{"update_id": 1, "message": {"message_id": 10, "chat": {"id": 123456789}, "text": "/start
alb2c3d4e5f60718"}}
// clique em botão
{"update_id": 2, "callback_query": {"id": "4382...", "data": "f:33333333-3333-4333-8333-
000000000001",
    "message": {"message_id": 11, "chat": {"id": 123456789}}}}

```

- chat.id é int (guarde em fazenda.telegram_chat_id; um chat só pode estar numa fazenda).
- callback_query.id vai para responder_clique.
- callback_query.data é o texto do botão, com no máximo 64 bytes (formatos na T6).

DiferencaLote: o que mudou entre o plano vigente e o candidato (seugado.contratos)

Campo	Tipo	Significado
lote_id, lote_nome	UUID, str	Lote afetado
antes	tuple[PassoPlano, ...]	Movimentações pendentes do lote no plano vigente
depois	tuple[PassoPlano, ...]	Movimentações do lote no plano novo

PassoPlano = data: date, piquete_destino_nome: str. Uma tupla vazia quer dizer “nenhuma movimentação”.

Formatos

PlanoManejo: o plano da semana

Tipo Python em seugado.contratos; em JSON via plano_para_dict / plano_de_dict; em TypeScript em frontend/src/lib/tipos.ts. Exemplo completo: tests/fixtures/plano_exemplo.json.

Campo	Tipo (Python)	Em JSON	Significado
id	UUID	texto	Id do plano

Campo	Tipo (Python)	Em JSON	Significado
fazenda_id	UUID	texto	Fazenda dona do plano
data_geracao	datetime (UTC, com fuso)	"2026-09-28T08:00:00+00:00"	Quando foi gerado
data_inicio	date	"2026-09-28"	Primeiro dia do plano (hoje)
horizonte_dias	int	7	Sempre 7
movimentacoes	tuple[Movimentacao, ...]	lista	Ordenadas por (data, lote_nome)
alertas	tuple[Alerta, ...]	lista	Ordenados por (data, tipo, texto)
pedidos_validacao	tuple[PedidoValidacao, ...]	lista	Ordenados pelo nome do piquete
piquetes	tuple[ResumoPiquete, ...]	lista	Um por piquete ativo, em ordem de nome

Movimentacao: “mover o lote X do piquete A para o B no dia D”

Campo	Tipo	Em JSON	Significado
id	UUID	texto	Id da movimentação; é o entidade_id do evento manejo_recomendado e o que vai nos botões do bot
data	date	"2026-09-28"	Dia da movimentação (sempre dia de manejo preferido)
lote_id / lote_nome	UUID / str	texto	Lote que muda
piquete_origem_id / piquete_origem_nome	UUID None / str None	texto ou null	De onde sai
piquete_destino_id / piquete_destino_nome	UUID / str	texto	Para onde vai
altura_destino_cm	float (1 casa)	número	Altura prevista do destino no início do dia
altura_entrada_alvo_cm	float	número	Altura ideal de entrada do capim do destino
altura_origem_cm	float None	número ou null	Altura prevista da origem no início do dia
altura_saida_alvo_cm	float None	número ou null	Altura de saída do capim da origem
dias_previstos	int	número	Dias que o lote deve ficar no destino
motivo	str	texto	Frase pronta, em português, para o produtor
confianca	Confianca	"alta" "media" "baixa"	Confiança da recomendação
motivo_confianca	str	texto	Frase pronta: o dado mais fraco por trás da recomendação

Alerta

Campo	Tipo	Em JSON	Significado
tipo	TipoAlerta	um de: sem_piquete_apto, capacidade_excedida, aguardando_parametro, estimativa_indisponivel, continuo_acima_maxima, continuo_abaixo_minima, lote_sem_piquete, sem_dia_de_manejo, passando_do_ponto	Tipo do aviso
data	date	"2026-10-01"	Dia a que se refere
texto	str	texto	Frase pronta para o produtor
confianca / motivo_confianca	Confianca / str	texto	Como na movimentação
piquete_id / lote_id	UUID None	texto ou null	A quem se refere

PedidoValidacao: pedido de medir o capim com régua

Campo	Tipo	Em JSON	Significado
piquete_id / piquete_nome	UUID / str	texto	Piquete a medir
motivo	str	texto	Frase pronta: por que o sistema está pedindo

ResumoPiquete: a fotografia de cada piquete (usada no mapa)

Campo	Tipo	Em JSON	Significado
piquete_id / nome	UUID / str	texto	Piquete
situacao	SituacaoPiquete	"ocupado" "descansando"	Tem gado ou está descansando
lote_atual_nome	str None	texto ou null	Lote que está nele
altura_hoje_cm	float None	número ou null	Altura estimada hoje; null = sem estimativa
altura_entrada_alvo_cm / altura_saida_alvo_cm	float None	número ou null	Alvos (nulos no contínuo ou sem parâmetro)
confianca / motivo_confianca	Confianca / str	texto	Confiança da estimativa do piquete
faltantes	tuple[str, ...]	lista de texto	O que falta para estimar ou planejar; vazio = tudo certo

2. Stack e onde fica o seu código

Python 3.12, FastAPI e `httpx` para falar com a API do Telegram (sem biblioteca de bot, ADR-022). Sem dependência nova.

Arquivos que são seus (só você edita):

- `src/seugado/delivery/mensagem.py`: textos (funções puras)
- `src/seugado/delivery/canais/base.py`: interface Canal
- `src/seugado/delivery/canais/telegram.py`: implementação Telegram
- `src/seugado/delivery/envio.py`: `enviar_plano`, `avisar_plano_candidato`
- `src/seugado/delivery/confirmacao.py`: registro das respostas e lembretes
- `src/seugado/delivery/bot.py`: a conversa (quem clicou em quê)
- `src/seugado/api/rotas_telegram.py`: webhook (já existe, vazio)
- `scripts/telegram_polling.py`
- `tests/delivery/*`

Você usa, mas não edita: `contratos.py`, `persistencia/eventos.py`, `persistencia/projecao_db.py`, `persistencia/planos.py` e `planner/comparacao.py` (do João), `jobs/ciclo.py` (do Leandro), `api/deps.py`.

Módulos que ainda não estão na sua branch. `executar_ciclo` (Leandro) e as funções de plano do João só chegam na `main` na integração de segunda. **Importe-os dentro das funções que os usam**, não no topo do arquivo, e nos testes injete módulos falsos com `monkeypatch.setitem(sys.modules, "seugado.jobs.ciclo", modulo_falso)` (idem para `seugado.persistencia.planos` e `seugado.planner.comparacao`). Para desenvolver com dados, use `tests/fixtures/plano_exemplo.json` COM `plano_de_dict`.

A interface `Canal` existe porque o WhatsApp vem depois (ADR-006): trocar de canal deve ser só escrever um novo adaptador.

3. Preparar o ambiente

Guia completo de instalação e comandos (Windows e Mac/Linux, token para o /docs, testes, Git e problemas comuns): `docs/equipe/COMO-RODAR.md`. Não rode migrações nem nada no Supabase: o banco já está pronto e é compartilhado.

Importante: o Python não lê o `.env` sozinho. Todo comando `uv run local` leva `--env-file .env`; sem isso os testes de banco são pulados.

1. Instale Git e **uv** (`docs.astral.sh/uv`). Clone e crie a branch `feat/leo-telegram`. `uv sync --group dev`.

2. **Crie dois bots** no Telegram com o **@BotFather** (`/newbot`):

- `SeuGado` (produção; o webhook vai apontar para o Render);
- `SeuGado Dev` (o seu, para testar local com polling).

Não dá para usar webhook e polling no mesmo bot ao mesmo tempo.

3. No BotFather, `/setcommands` nos dois bots: `plano` - Ver o plano da semana · `alturas` - Ver e corrigir a altura dos piquetes · `mover` - Registrar que mudei um lote de piquete · `ajuda` - Como usar.

4. `.env` (a partir do `.env.example`): `DATABASE_URL` (o Kauê manda em privado), `TELEGRAM_BOT_TOKEN` e `TELEGRAM_BOT_USERNAME` (do bot **Dev**, localmente), `TELEGRAM_WEBHOOK_SECRET` (texto aleatório longo). Mande os dados do bot de **produção** ao Leandro, em privado, para o deploy.
5. Rode local com polling: `uv run --env-file .env python scripts/telegram_polling.py`.

Antes do PR: `uv run ruff check .`, `uv run mypy` e `uv run --env-file .env pytest` sem erro.

4. A regra de gravação do projeto

Toda resposta do produtor vira **evento**, sempre com `ocorrido_em = datetime.now(UTC)` (a data real vai **dentro** do payload), seguido de `reconstruir_projecao` e `commit`:

```
registrar_evento(conn, fazenda_id, TipoEvento.MANEJO_CONFIRMADO, OrigemEvento.PRODUTOR,
                 ocorrido_em=datetime.now(UTC), payload={...},
                 ator=f"telegram:{chat_id}", chave_idempotencia=f"resposta:{mov.id}")
reconstruir_projecao(conn, fazenda_id)
conn.commit()
```

Payloads (todos os campos obrigatórios; UUIDs e datas como texto ISO):

```
# Fiz → manejo_confirmado (entidade_id = id da movimentação)
{"entidade_id": mov.id, "lote_id": mov.lote_id, "piquete_destino_id":
mov.piquete_destino_id,
  "data_execucao": "2026-09-28"}
# Não fiz → manejo_recusado
{"entidade_id": mov.id, "motivo": None}
# Fiz diferente → manejo_divergente
{"entidade_id": mov.id, "lote_id": mov.lote_id, "piquete_real_id": "<uuid>",
  "data_execucao": "2026-09-29", "observacao": None}
# Movimentação avulsa → manejo_confirmado com entidade_id NOVO (uuid4), sem chave
{"entidade_id": "<uuid4>", "lote_id": "<uuid>", "piquete_destino_id": "<uuid>",
  "data_execucao": "2026-09-29"}
# Altura medida pelo bot → altura_medida
{"entidade_id": "<uuid4>", "piquete_id": "<uuid>", "data": "2026-09-29",
  "altura_cm": 35.0, "meio": "bot"}
```

`chave_idempotencia = f"resposta:{mov.id}"` em Fiz, Não fiz e Fiz diferente: garante **uma resposta por movimentação**, mesmo com dois toques.

Tabelas que você lê ou escreve:

Tabela	Uso
fazenda	id, nome, timezone, telegram_chat_id, codigo_vinculo_telegram. Você faz UPDATE fazenda SET telegram_chat_id no /start
estado_lote (derivada)	lote_id, nome, piquete_atual_id
estado_piquete (derivada)	piquete_id, nome, ativo, lote_atual_id
telegram_conversa	chat_id (PK), estado, dados (jsonb), atualizado_em: em que passo da conversa o produtor está

5. Tarefas (na ordem)

T1: Interface e canal Telegram (`canais/base.py`, `canais/telegram.py`)

Por quê: isola o Telegram do resto. O WhatsApp entra depois só como outro adaptador.

Como:

```
@dataclass(frozen=True, slots=True)
class Botao:
    texto: str
    dados: str          # callback_data: no máximo 64 bytes

class Canal(Protocol):
    def enviar_texto(self, chat_id: int, texto: str,
                    botoes: Sequence[Sequence[Botao]] = ()) -> None: ...
    def responder_clique(self, id_clique: str, texto: str | None = None) -> None: ...
```

`CanalTelegram(token):`

- `enviar_texto` → POST `https://api.telegram.org/bot<token>/sendMessage`, COM `{"chat_id", "text", "parse_mode": "HTML", "reply_markup": {"inline_keyboard": [{"text", "callback_data"}]}}` (sem `reply_markup` quando não há botões). Use `httpx.post(..., timeout=15)` e `raise_for_status()`.
- `responder_clique` → `answerCallbackQuery`.
- `canal_padrao()` lê `TELEGRAM_BOT_TOKEN`.
- Nomes dentro de texto HTML passam por `html.escape`.
- `Botao.dados` com mais de 64 bytes → `ValueError`.

Critérios de aceite:

- [] Testes com `httpx.post` falso conferem URL, corpo e teclado.
- [] Nenhum teste chama o Telegram de verdade.

T2: Textos (`mensagem.py`, `puro`)

Por quê: é o que o produtor lê, e é a única coisa que ele vê do SeuGado. Regras: **sempre o motivo junto da ordem**, números em **cm** (nunca kg, NDVI ou termos técnicos), **um plano por mensagem**.

Como:

```
def texto_plano(plano: PlanoManejo, fazenda_nome: str, atualizado: bool = False) -> str
def botoes_plano(plano: PlanoManejo) -> list[list[Botao]]
def texto_movimentacao(mov: Movimentacao) -> str
def botoes_movimentacao(mov: Movimentacao) -> list[list[Botao]]
def texto_alturas(plano: PlanoManejo) -> str
def botoes_alturas(plano: PlanoManejo) -> list[list[Botao]]
def texto_diferencas(diferencas: Sequence[DiferencaLote]) -> str
def texto lembrete(pendentes: Sequence[Movimentacao]) -> str
```

Formato de `texto_plano` para `plano_exemplo.json` (é a referência; seus testes conferem estas linhas):

 SeuGado – Fazenda Exemplo
Plano da semana: seg 28/09 a dom 04/10

 Segunda, 28/09
▶ Mover Recria: Piquete 2 → Piquete 6
0 Piquete 2 chegaria a 11,6 cm antes do próximo dia de manejo (qui 01/10), abaixo da saída de 15 cm. 0 Piquete 6 está em 34 cm (ponto de entrada: 30 cm).
Previsão: 7 dias no piquete · Confiança: média
<i>(a altura de entrada do Marandu no pastejo rotacionado vem de fonte com confiança média)</i>

▶ Mover Vacas com bezerro: Piquete 5 → Piquete 1
...

 Quinta, 01/10
▶ Mover Vacas com bezerro: Piquete 1 → Piquete 3
...

 Avisos
• 0 Piquete 8 (Mombaça) ainda não recebe recomendação: ...
• 0 Piquete 7 (pastejo contínuo) está em 38 cm, ...

 Medições pedidas
• Piquete 4: última imagem de satélite sem nuvem há 18 dias. Meça a altura com uma régua.

Toque abaixo para dizer o que você fez, conferir as alturas ou mandar uma medição.

- atualizado=True → primeira linha  Plano atualizado – {fazenda}.
- Sem movimentações → "Nenhuma movimentação necessária nesta semana."
- Seções sem itens não aparecem.
- Limite de 4.096 caracteres: corte os avisos e termine com "... (mais avisos omitidos)".
- Dia por extenso Segunda ... Domingo; abreviado seg ter qua qui sex sáb dom; confiança alta / média / baixa.

botoes_plano, uma linha por botão:

- por movimentação: "{n}. {ddd dd/mm} · {lote} → {destino}", dados "m:{mov.id}";
- por pedido de medição: " Informar altura do {piquete}", dados "a:{piquete_id}";
- no fim: " Conferir alturas dos piquetes", dados "h:".

botoes_movimentacao:  Fiz ("f:{id}"),  Não fiz ("n:{id}"),  Fiz diferente ("d:{id}").

texto_alturas: uma linha por piquete do plano.piquetes:

- "• Piquete 1: ~31,5 cm (confiança alta)";
- altura_hoje_cm nulo → "• Piquete 8: sem estimativa".

Termina com “Se alguma estiver diferente do que você vê no pasto, toque no piquete e mande a altura medida com régua. Quando terminar, toque em Refazer o plano.” botoes_alturas: um botão por piquete, "Corrigir {piquete}" → "a:{piquete_id}", e no fim " Terminei – refazer o plano" → "r:".

texto_diferencas, por lote:

▶ Recria
antes: qui 01/10 → Piquete 3
agora: qui 01/10 → Piquete 4

Lista vazia → antes: nenhuma movimentação (OU agora: ...).

texto lembrete: “Você ainda não me disse se fez estas movimentações:” + uma linha por movimentação (“• seg 28/09 • Recria → Piquete 6”). Os botões são os “m:{id}”.

Critérios de aceite:

- [] texto_plano(plano_exemplo, "Fazenda Exemplo") reproduz as linhas acima.
- [] Nenhum texto contém “NDVI”, “kg” ou “MS”.
- [] Todos os dados de botão com ≤ 64 bytes.

T3: Envio (envio.py)

Por quê: é a porta que a rotina do Leandro usa.

Como:

1. enviar_plano(conn, plano, atualizado=False, canal=None):
 - SELECT nome, telegram_chat_id FROM fazenda WHERE id = %s; sem chat → False;
 - senão, canal.enviar_texto(chat_id, texto_plano(...), botoes_plano(plano)) → True.
2. avisar_plano_candidato(conn, plano, diferencas, canal=None): sem chat → False. Senão, manda “📡 Chegaram imagens novas do satélite e o plano da semana mudou para {n} lote(s). Você pode manter o seu plano ou ver as mudanças.”, com os botões 👁 Ver mudanças (“pv:{plano.id}”) e 👍 Manter meu plano (“pk:{plano.id}”).

Critérios de aceite:

- [] Sem chat vinculado → False, sem chamada HTTP.

T4: Respostas e lembretes (confirmacao.py)

Por quê: é o que fecha o ciclo. Sem saber o que o produtor fez, o próximo plano parte de uma fazenda que não existe. E **só ele confirma:** por isso o lembrete.

Como:

```
def resposta_existente(conn, fazenda_id: UUID, mov_id: UUID) -> str | None
def registrar_fiz(conn, fazenda_id, mov: Movimentacao, hoje: date, ator: str) -> None
def registrar_ao_fiz(conn, fazenda_id, mov: Movimentacao, ator: str) -> None
def registrar_diferente(conn, fazenda_id, mov: Movimentacao, piquete_real_id: UUID,
data_execucao: date, ator: str) -> None
def registrar_avulsa(conn, fazenda_id, lote_id: UUID, piquete_id: UUID, data_execucao:
date, ator: str) -> None
def registrar_altura(conn, fazenda_id, piquete_id: UUID, altura_cm: float, hoje: date,
ator: str) -> None
def lembrar_pendentes(conn, fazenda_id: UUID, hoje: date, canal: Canal | None = None) ->
int
```

1. registrar_fiz: data_execucao = min(mov.data, hoje) (se tocou antes do dia, fez hoje; se depois, vale o dia do plano).
2. Todas as funções de registro chamam reconstruir_projecao, mas **não** fazem commit: o bot.py faz.

3. `registrar_diferente` e `registrar_avulsa`: o piquete precisa existir, estar ativo, ser diferente do atual do lote e estar **vazio** (`estado_piquete.lote_atual_id IS NULL`), e `data_execucao` não pode ser futura. Dois lotes nunca dividem um piquete no MVP.
4. `registrar_altura`: `0 < altura_cm <= 400`.
5. `lembrar_pendentes`:
 - pegue o plano vigente (`carregar_plano_atual`) e as movimentações com `mov.data < hoje` **não** respondidas (`ids_respondidos`);
 - se houver alguma e a fazenda tiver chat, envie `texto lembrete` com os botões `"m:{id}"`;
 - devolva quantas eram. **Não** registra nada: enquanto o produtor não responde, o lote continua onde estava.

Critérios de aceite:

- [] Tocar “Fiz” duas vezes gera um único evento.
- [] `lembrar_pendentes` não grava nenhum evento.
- [] “Fiz diferente” não oferece nem aceita piquete ocupado.

T5: Plano candidato: ver mudanças, manter ou trocar (no `bot.py`)

Por quê: o produtor pode já ter se planejado com o plano que recebeu. Ele decide se quer ver o que mudou e, vendo, se fica com o antigo ou troca pelo novo.

Como: cliques:

- `pv:<plano_id>` (“Ver mudanças”):
 - se `status_do_plano != "candidato"`, responda *“Essa atualização não vale mais.”*;
 - senão, calcule `comparar_planos(carregar_plano_atual(...), carregar_plano(plano_id), ids_respondidos(...), hoje)` e mande `texto_diferencas(...)` com os botões  Usar o plano novo (`"pu:<id>"`) e  Manter o anterior (`"pk:<id>"`).
- `pu:<plano_id>`: se ainda `candidato` → `promover_candidato` → `commit` → `enviar_plano(conn, plano, atualizado=True)`.
- `pk:<plano_id>`: se ainda `candidato` → `descartar_plano` → `commit` → *“Combinado, seguimos com o plano que você já tem.”*

Critérios de aceite:

- [] Ver → Usar: o plano novo vira vigente e chega completo.
- [] Manter: nada muda e o candidato fica descartado.
- [] Clicar num candidato velho responde “não vale mais”.

T6: A conversa (no `bot.py`)

Por quê: é onde os toques viram eventos, sem o produtor digitar nada além de números.

Como: `processar_update(conn, update: dict, canal: Canal, hoje: date) -> None`.

Identificação:

- `chat_id` de `update["message"]["chat"]["id"]` ou de `update["callback_query"]["message"]["chat"]["id"]`.

- Fazenda: `SELECT id, nome FROM fazenda WHERE telegram_chat_id = %s`.
- Sem fazenda, só `/start <codigo>` funciona; o resto responde *“Este chat ainda não está ligado a uma fazenda. Peça o link à equipe SeuGado.”*

Mensagens de texto:

Texto	Ação
<code>/start <codigo></code>	<code>UPDATE fazenda SET telegram_chat_id = chat_id WHERE codigo_vinculo_telegram = codigo</code> → <i>“Pronto! A {fazenda} está conectada. Você vai receber o plano da semana aqui.”</i> Código inválido, ou chat já ligado a outra fazenda → mensagem amigável
<code>/plano</code>	manda <code>texto_plano</code> + <code>botoes_plano</code> do plano vigente (ou <i>“Ainda não há plano.”</i>)
<code>/alturas</code>	manda <code>texto_alturas</code> + <code>botoes_alturas</code> do plano vigente
<code>/mover</code>	inicia a movimentação avulsa
<code>/ajuda</code>	explica os comandos e os botões
número (35, 35,5, 35 cm) com a conversa no estado <code>altura</code>	<code>registrar_altura</code> → <code>commit</code> → <i>“Anotado: {piquete} com 35 cm hoje.”</i> e manda de novo <code>texto_alturas</code> + <code>botoes_alturas</code> (estado limpo)
qualquer outro	<i>“Não entendi. Use /plano, /alturas, /mover ou toque nos botões.”</i>

Cliques (sempre chame `responder_clique` primeiro):

Dados	Ação
<code>m:<id></code>	acha a movimentação no plano vigente e manda <code>texto_movimentacao</code> + <code>botoes_movimentacao</code> . Não achou → <i>“Esta movimentação é de um plano antigo. Use /plano.”</i> Já respondida → <i>“Você já respondeu esta movimentação.”</i>
<code>f:<id></code>	<code>registrar_fiz</code> → <code>commit</code> → <i>“ Anotado: {lote} no {destino}.”</i>
<code>n:<id></code>	<code>registrar_ao_fiz</code> → <code>commit</code> → <i>“Entendido. Vou refazer o plano...”</i> → recalcular
<code>d:<id></code>	<i>“Para qual piquete o {lote} foi?”</i> , com os piquetes ativos e vazios (menos a origem) → <i>“Quando?”</i> (Hoje, Ontem, Anteontem) → <code>registrar_diferente</code> → <code>commit</code> → recalcular
<code>a:<piquete_id></code>	<code>estado altura</code> → <i>“Digite a altura do {piquete} em centímetros (só o número).”</i>
<code>h:</code>	manda <code>texto_alturas</code> + <code>botoes_alturas</code>
<code>r:</code>	<i>“Refazendo o plano com as alturas novas...”</i> → recalcular
<code>pv: / pu: / pk:</code>	T5
<code>o:<n></code>	a opção número <code>n</code> do passo atual da conversa (abaixo)

Conversa com vários passos (`telegram_conversa`, um registro por chat, sobrescrito a cada passo com `INSERT ... ON CONFLICT (chat_id) DO UPDATE`):

estado	dados	Próximo passo
<code>divergente_piquete</code>	<code>{"mov_id", "opcoes": [piquete_ids]}</code>	<code>o:<n></code> → <code>divergente_data</code>

estado	dados	Próximo passo
divergente_data	{"mov_id", "piquete_id"}	o:0/1/2 = hoje/ontem/anteontem → registrar
mover_lote	{"opcoes": [lote_ids]}	o:<n> → mover_piquete
mover_piquete	{"lote_id", "opcoes": [piquete_ids]} (só ativos e vazios)	o:<n> → mover_data
mover_data	{"lote_id", "piquete_id"}	o:0/1/2 → registrar_avulsa → commit → recalcular
altura	{"piquete_id"}	texto numérico → registrar_altura → commit

Recalcular: mande “*Recalculando o plano...*” e chame `executar_ciclo(conn, fazenda_id, ingerir_satelite=False, atualizado=True)`. Ele salva o plano novo como vigente e envia com o cabeçalho “Plano atualizado”. Se falhar: “*Não consegui recalcular agora. Tento de novo na próxima rotina.*” e registre no log.

Critérios de aceite:

- [] Com `canal` falso e banco de teste (pulando se `SEUGADO_TEST_DATABASE_URL` não existir):
 - `/start` vincula;
 - `f`: gera `manejo_confirmado`;
 - `d`: → `o:1` → `o:0` gera `manejo_divergente` e chama o recálculo (monkeypatch);
 - `a`: → 28 gera `altura_medida`, e `r`: chama o recálculo.
- [] Nenhuma exceção sobe sem resposta ao produtor.

Fora do escopo: IA ou linguagem natural para interpretar texto; imagens e mapas no bot (pós-MVP); vários usuários por fazenda; WhatsApp.

T7: Webhook e polling (`rotas_telegram.py`, `scripts/telegram_polling.py`)

Como:

1. `POST /telegram/webhook`:
 - confere `X-Telegram-Bot-API-Secret-Token` (diferente → 403);
 - agenda `processar_update` em `BackgroundTasks`, com **conexão própria** aberta e fechada dentro da tarefa;
 - responde `{"ok": true}` na hora.

hoje = data atual em `America/Fortaleza`.
2. `scripts/telegram_polling.py`: `deleteWebhook`; em laço, `getUpdates` com `offset` e `timeout=30`; para cada update, abre conexão, chama `processar_update` e fecha.
3. Depois do deploy do Leandro, registre uma vez:

`https://api.telegram.org/bot<TOKEN>/setWebhook?url=https://<api>.onrender.com/telegram/webhook&secret_token=<SEGREDO>`. Confira com `getWebhookInfo`.

Critérios de aceite:

- [] Sem o cabeçalho certo → 403.
- [] O webhook responde em < 1 s mesmo quando o recálculo demora.

T8: Testes e PR

- `tests/delivery/test_mensagem.py` (sem banco, com a fixture);
- `test_telegram.py` (HTTP falso);
- `test_confirmacao.py` e `test_bot.py` (banco de teste).

Depois, `ruff`, `mypy`, `pytest` e o PR `feat/leo-telegram` → `main`, com prints do bot.

6. Roteiro da demo (terça)

1. A equipe manda o link do Telegram ao “produtor” → `/start` → “Pronto!”
2. No painel, “Gerar e enviar plano agora” → o plano chega no celular.
3. `/alturas` → corrigir o Piquete 4 para 28 → “Terminei” → chega o **plano atualizado**.
4. Tocar na movimentação de segunda → “Fiz diferente” → outro piquete → “Hoje” → plano atualizado de novo.
5. Mostrar (rodando a rotina diária com imagem nova) a pergunta “quer ver as mudanças?”.

7. Usando IA no seu fluxo

Dê à IA este documento, `src/seugado/contratos.py` e `tests/fixtures/plano_exemplo.json`, e peça uma tarefa por vez:

“Leia `docs/equipe/LEO.md` (seções ‘Entradas e saídas’, 4 e a tarefa T2) e `contratos.py`. Implemente `src/seugado/delivery/mensagem.py` com funções puras que reproduzam o formato do exemplo. Não altere outros arquivos nem adicione dependências.”

Se a IA quiser usar `python-telegram-bot`, IA para interpretar texto, ou mexer em arquivo de outra pessoa, a resposta é não: fale com o Kauê.